

Deferred Rendering Techniques

Concepts, implementation details, limitations, and tuning

- Shadow mapping
- SSAO
- SSR
- Image based lighting
- Transparency
- Refraction

How to read this guide

Concept first, then the exact project behavior and its engineering tradeoffs

THE MENTAL MODEL

Each technique answers a different visibility question:

- **Shadow maps:** can this light see this surface point?
- **SSAO:** is nearby geometry likely to block ambient light?
- **SSR:** where does a reflected ray first meet visible opaque depth?
- **IBL:** what light arrives from the surrounding environment?
- **Transparency:** how should ordered surface layers be composited?
- **Refraction:** where does a transmitted ray sample the opaque scene?

THREE LEVELS ON EVERY TOPIC

1

Concept

The geometric or shading idea, with diagrams and equations.

2

Implementation

What the current C++ and GLSL actually do, including pass order and data.

3

Limits

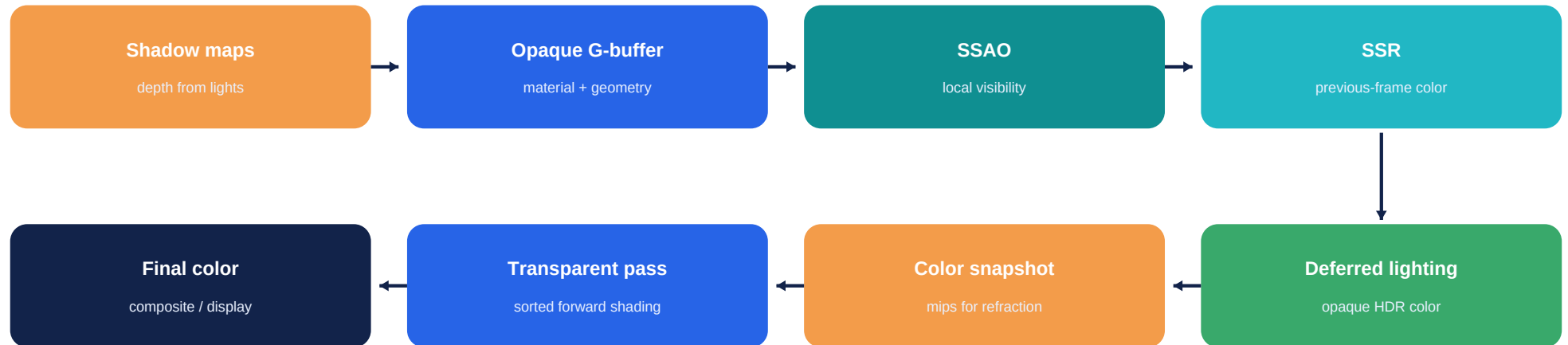
Failure modes, tuning controls, and where a more advanced renderer would differ.

NOTATION AND SCOPE

- World-space vectors are uppercase: N normal, V view, L light, R reflection.
- Depth tests refer to the opaque G-buffer unless a shadow-map depth texture is explicitly named.
- The phrase **deferred renderer** includes the final sorted forward pass used for transparent materials.
- Implementation facts are traced to the current files listed on page 31; older recap documents are not treated as the source of truth.

The hybrid frame pipeline

Deferred for opaque surfaces; sorted forward shading for transparent ones



WHY DEFER LIGHTING?

- Opaque geometry writes material and geometric attributes once. Lighting then evaluates only the closest visible surface per pixel.
- Fullscreen SSAO and SSR can reuse the same position, normal, material, and depth buffers.
- The cost shifts from geometry-times-lights toward pixels-times-lights, which is attractive for many dynamic lights.

WHY A FORWARD TRANSPARENT PASS?

- A single G-buffer pixel cannot retain an arbitrary stack of transparent layers.
- Transparent surfaces therefore shade after opaque lighting, depth-test against opaque depth, and blend into the HDR color target.
- Refraction requires a stable copy of opaque color so a fragment never reads and writes the same target.

G-buffer anatomy

The opaque geometry pass materializes the first visible hit for later fullscreen work

Albedo

RGB surface color

Normal

encoded world normal

Position

world-space hit point

Material

roughness / metallic / reflectance

Emission

self-emitted radiance

Depth

opaque depth test and reconstruction

CONSUMERS

SSAO

normal + position + depth

SSR

normal + position + material + depth

Lighting

all attachments

Refraction

opaque depth + position + HDR color

IMPORTANT BOUNDARY

Only opaque and alpha-masked mesh instances enter these attachments. Blended or transmissive meshes are deliberately absent, so they cannot contribute to SSAO, opaque SSR hits, or opaque debug buffers. This boundary is the defining tradeoff of the current hybrid architecture.

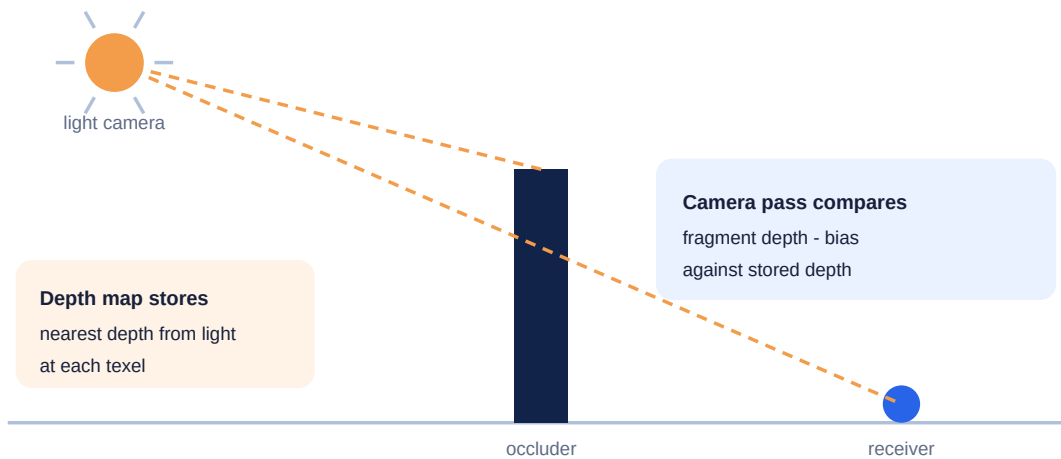
SECTION 01

Shadow mapping

Render depth from the light, then ask whether the camera-visible fragment is farther from that light than the stored blocker.

Depth from the light's point of view

A visibility lookup, not a lighting model



CORE TEST

```
lightUVZ = project(lightVP, worldPos)
closest = shadowMap(lightUVZ.xy)
visible = lightUVZ.z - bias <= closest

visibility = average(nearby tests)
```

- The shadow texture records only the nearest surface seen by the light.
- The camera pass projects each shaded point into that texture and compares depths.
- Percentage-closer filtering (PCF) averages neighboring binary comparisons to soften texel edges.

BIAS: NECESSARY BUT IMPERFECT

Finite precision and mismatched raster samples can make a surface shadow itself. A positive bias suppresses this **shadow acne**. Too much bias detaches the shadow from the caster, producing **peter-panning**. The project uses a constant radial bias for local lights and a normal-dependent bias for distant lights.

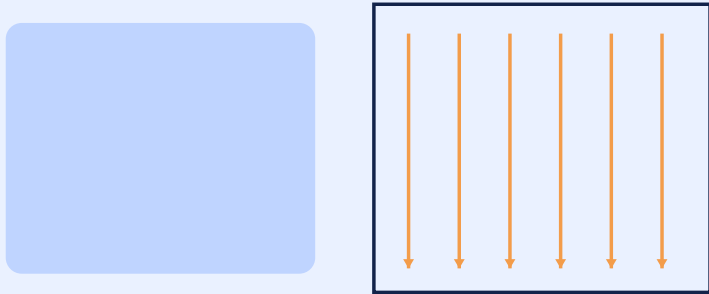
WHAT THE MAP CANNOT KNOW

- A single depth layer cannot represent translucent attenuation or multiple blockers.
- Resolution is finite, so projected texel size controls detail and shimmering.
- PCF smooths the test but does not model a physical area-light penumbra.

Two shadow-map families in this renderer

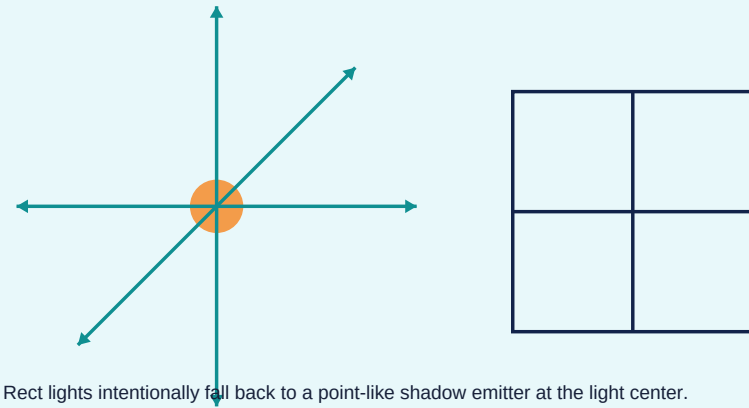
Distant lights use an orthographic array; local lights use cubemap-array layers

DISTANT LIGHT



- Scene bounds are transformed into light space and fitted by an orthographic projection with XY padding.
- Each selected light owns one layer of a `sampler2DArray`.
- Lighting applies a 3x3 PCF kernel and a slope-aware bias based on $\text{dot}(\mathbf{N}, \mathbf{L})$.

SPHERE AND RECT LIGHTS



- Rect lights intentionally fall back to a point-like shadow emitter at the light center.
- Six 90-degree views are stored in a cubemap array; radial depth is normalized by each caster's far plane.
- A 20-direction PCF-style kernel grows slightly with distance from the light.

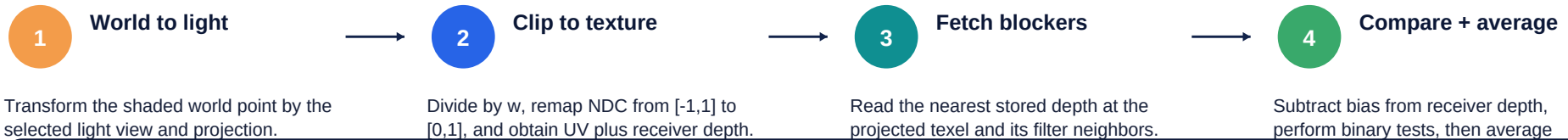
SELECTION AND LIFETIME

- Up to eight eligible shadow casters are selected. Distant lights receive a large priority bonus; local lights are ranked by emitted power and relevance to the camera or scene center.
- Automatic local far planes enclose the scene bounds from the light position. User-provided light shadow radius or global shadow far values override that fit.
- Only opaque draw-list instances are rendered into shadow maps. Blended and transmissive surfaces neither cast colored shadows nor attenuate light.

From world position to filtered visibility

Projection defines the lookup; filtering and bias define how reliable it appears

THE FOUR COORDINATE STEPS



```

projected = lightVP * vec4(worldPos, 1)
uvz = projected.xyz / projected.w * 0.5 + 0.5
visible_i = (uvz.z - bias <= storedDepth_i)
visibility = sum(visible_i) / sampleCount
  
```

PROJECTED DEPTH

Distant lights use orthographic projected depth. Their rays are parallel, so XY chooses a texel and Z orders blockers along one shared light direction. Tight scene-bound fitting spends the available depth and texture resolution where geometry exists.

RADIAL CUBEMAP DEPTH

Local lights store distance from the light divided by a far plane. The lookup direction chooses one of six faces; the normalized distance is compared with the receiver's radial distance. A loose far plane compresses useful precision into a smaller portion of [0,1].

WHAT PCF MEANS

Each tap still answers only lit or shadowed. A 3x3 kernel can return 0/9 through 9/9 visible taps, turning a one-texel step into a short gray transition. This is antialiasing of the depth test; its apparent softness follows texel footprint and filter radius, not the physical emitter size.

Reading the shadow diagnostic

The debug view reports average visibility, not raw depth-map faces



Dining-room shadow-factor capture. White means visible to selected casters; darker regions are shadowed after PCF.

TUNING CHECKLIST

- Raise `ShadowMapResolution` when projected texels are visibly coarse.
- Reduce `ShadowBias` until acne appears, then increase slightly.
- Keep local far planes tight; excessive range wastes radial depth precision.
- Watch distant-light edges while moving the camera: the current fit has no cascade or texel snapping.
- Remember that the debug value averages only lights that own a shadow map.

Expected artifacts: acne from insufficient bias; peter-panning from too much bias; shimmering from moving projections; hard area-light behavior because sphere and rect lights use point-like depth origins; and missing shadows for transparent casters.

SECTION 02

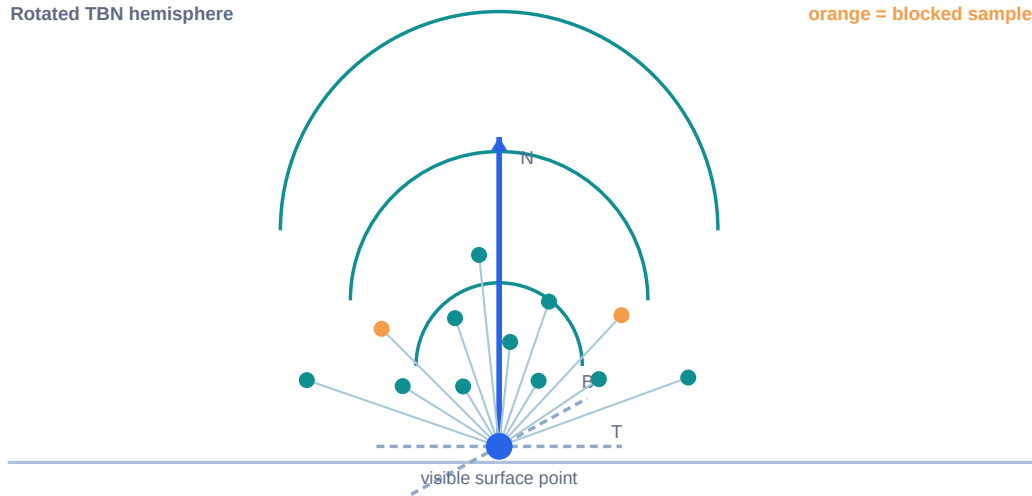
Screen Space Ambient Occlusion

Estimate local ambient visibility from the opaque G-buffer, then blur without crossing strong depth or normal edges.

A rotated hemisphere around each visible point

A local screen-space approximation to ambient visibility

Rotated TBN hemisphere



CURRENT SHADER

- A deterministic 32-vector hemisphere kernel is generated on the CPU. Samples become denser near the origin through a quadratic scale ramp.
- A tiled 4x4 noise texture rotates the tangent frame per pixel, breaking up directional banding.
- Samples are transformed into view space, projected to UV, and compared with fetched G-buffer positions.
- Radius controls neighborhood size; bias prevents the origin surface from counting itself; a smooth range check rejects distant depth layers.

EDGE-AWARE 3x3 BLUR

Raw AO is filtered with spatial, depth, and normal weights. This reduces kernel noise while resisting halos across silhouettes. Disabling blur copies the raw factor. Background pixels resolve to $AO = 1$.

HOW LIGHTING USES IT

AO multiplies ambient and diffuse environment light directly. It lightly modulates direct diffuse and specular environment or SSR terms. This is an artistic integration, not a full indirect-light transport model.

What one kernel sample actually does

Rotate, offset, project, fetch, compare, and range-weight

ONE SAMPLE, END TO END

1

Orient

Use normal N plus the tiled random vector to build a TBN frame; rotate a hemisphere kernel vector into view space.

2

Offset

$\text{samplePosVS} = \text{fragPosVS} + \text{rotatedSample} * \text{radius}$. The radius is measured in the same view-space units as position.

3

Project

Apply the camera projection, divide by w , and remap XY to texture UV . Out-of-range UV contributes no occlusion.

4

Compare

Fetch the G-buffer position at UV , transform it to view space, and ask whether it lies in front of the intended sample.

OCCUSION AND RANGE

```
blocked = fetchedZ >= sampleZ + bias
range = smoothstep(0, 1, radius / abs(fragZ-fetchedZ))
A0 = 1 - sum(blocked * range) / kernelSize
```

The sign convention follows this renderer's view-space Z . Bias prevents the source surface from winning its own comparison; the smooth range term stops unrelated depth layers from darkening the pixel.

WHY THE BLUR IS EDGE-AWARE

```
weight = spatialWeight
* depthWeight(centerZ, neighborZ)
* normalWeight(centerN, neighborN)
```

A plain blur would mix foreground AO with background or another surface, creating dark outlines. Depth similarity preserves silhouettes; normal similarity preserves creases. The 3×3 support is intentionally small, so it reduces noise without erasing all contact detail.

From raw visibility to contact grounding

Full-resolution output, optional edge-aware blur, no temporal accumulation



Dining-room AO factor. White is unoccluded; darker creases and contacts reduce ambient contribution.

PARAMETER EFFECTS

- **Radius:** world/view-space reach. Too large produces broad halos and scale mismatch.
- **Bias:** depth separation before a sample counts as blocking. Too low self-occludes flat surfaces.
- **Kernel size:** 4-32 samples. More samples reduce noise and increase bandwidth/ALU cost.
- **Intensity:** remaps the final AO factor in lighting; it does not alter sampling.
- **Blur:** stabilizes noise but can soften very fine contact detail.

Screen-space limitation: only currently visible opaque surfaces participate. Geometry behind the first depth layer, beyond the viewport, or represented by transparency is unknown. There are no bent normals, multi-bounce AO, half-resolution optimization, or temporal history rejection.

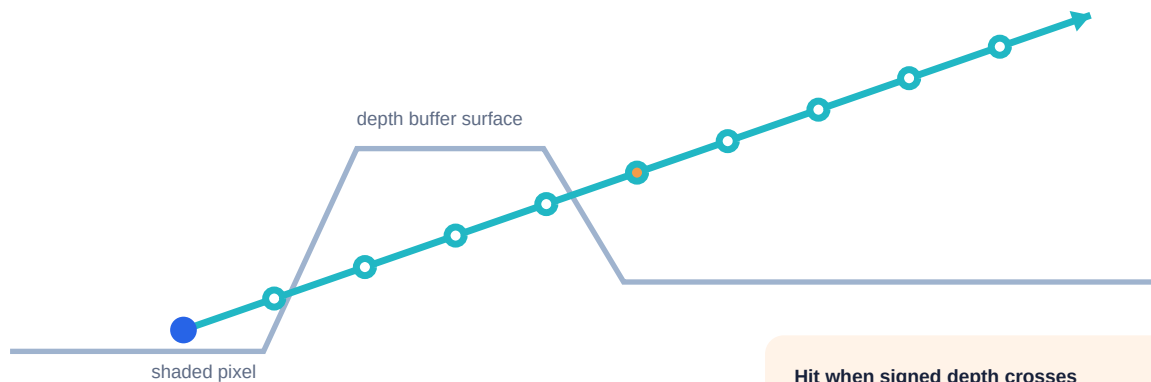
SECTION 03

Screen Space Reflections

Project a reflected ray through screen pixels, detect a crossing against opaque depth, and attach a confidence to the result.

Tracing reflection against the depth buffer

Fast and coherent because the ray sees the same first-hit representation as the camera



Hit when signed depth crosses
then refine the bracket 5 times

PER-PIXEL START

```
V = normalize(camera - pos)
R = normalize(reflect(-V, N))

if roughness < maxRoughness:
    trace(pos, R, maxDistance)
```

- The material roughness gate avoids expensive rays that would require broad glossy integration.
- A start bias in pixels skips the origin neighborhood and reduces self-intersections.
- The result buffer stores reflected RGB plus confidence in alpha.

WHY PREVIOUS-FRAME COLOR?

SSR runs before the current deferred-lighting pass, but it needs already shaded color at the hit. The renderer therefore samples a separate HDR source texture copied at the end of the previous normal-color frame. This avoids a read/write feedback hazard, but moving cameras and objects can reveal one-frame lag or stale color.

The shared screen-space tracer

Perspective-correct pixel stepping with a refined signed-depth crossing

1 Clip interval

Transform origin and endpoint to view/clip space; clip rays that approach the camera plane.

2 Pixel march

Walk the dominant projected axis using pixel stride and perspective-correct Q/k interpolation.

3 Depth bracket

Sample opaque position and detect a sign change in $\text{sceneZ} - \text{rayZ}$ across two valid endpoints.

4 Refine

Binary-search the crossing five times. Thickness validates only the final residual.

5 Stability

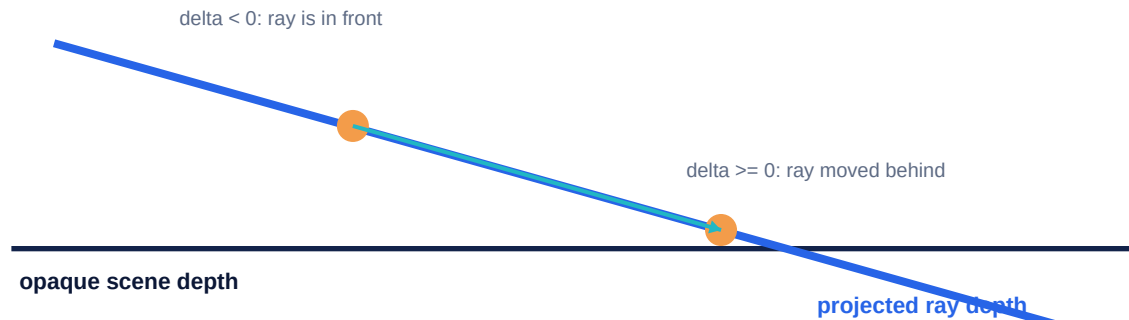
Compare center depth with four neighbors. Depth discontinuities reduce confidence or reject the hit.

Statuses distinguish hit, unstable, off-screen, no-hit, and no-displacement. SSR accepts only stable hits; refraction uses the status to choose opaque-color or environment fallbacks.

A depth crossing is only a candidate

Refinement identifies a plausible hit; confidence decides how strongly it may contribute

SIGNED-DEPTH BRACKET



A sign change brackets where the projected ray passes through the first opaque depth layer. Binary refinement narrows that interval, but the result can still lie on a silhouette, thin object, or discontinuity. The final thickness and neighborhood-depth checks decide whether it is usable.

CONFIDENCE IS A PRODUCT

```
confidence = roughnessFade
* edgeFade
* distanceFade
* depthStability
```

Each term describes a different source of trust. Multiplication makes any decisive failure dominate: a geometrically plausible crossing near the viewport edge can still fade out because its reflected neighborhood is about to leave the available image.

COMPOSITION

```
ssrSpecular = hitColor * Fresnel
* confidence * SSRIntensity
finalSpecular = environmentIBL + ssrSpecular
```

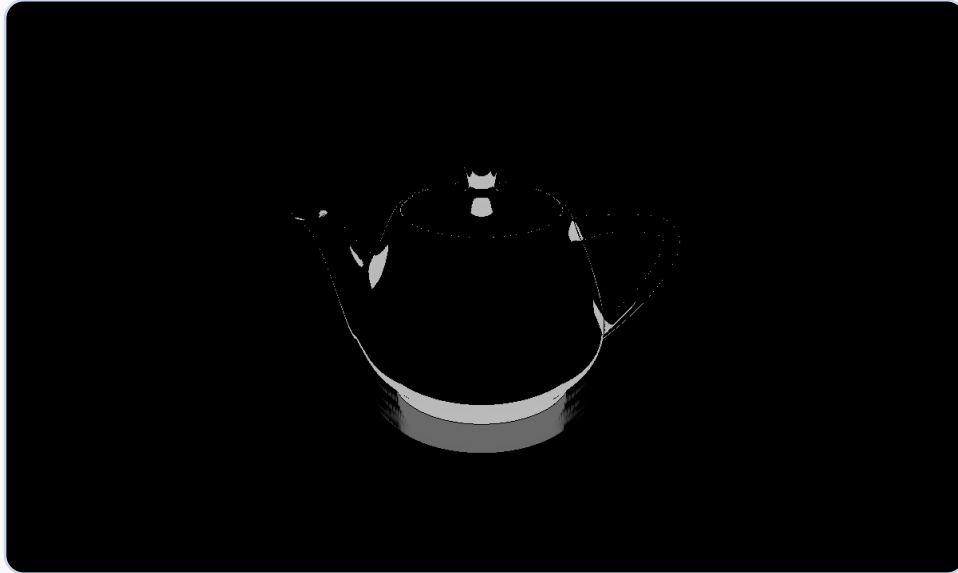
Fresnel preserves view- and material-dependent reflectivity after the tracer finds color. Roughness already affects eligibility and confidence. In this implementation SSR adds local scene detail on top of IBL rather than replacing the environment term.

HISTORY AND MISS SEMANTICS

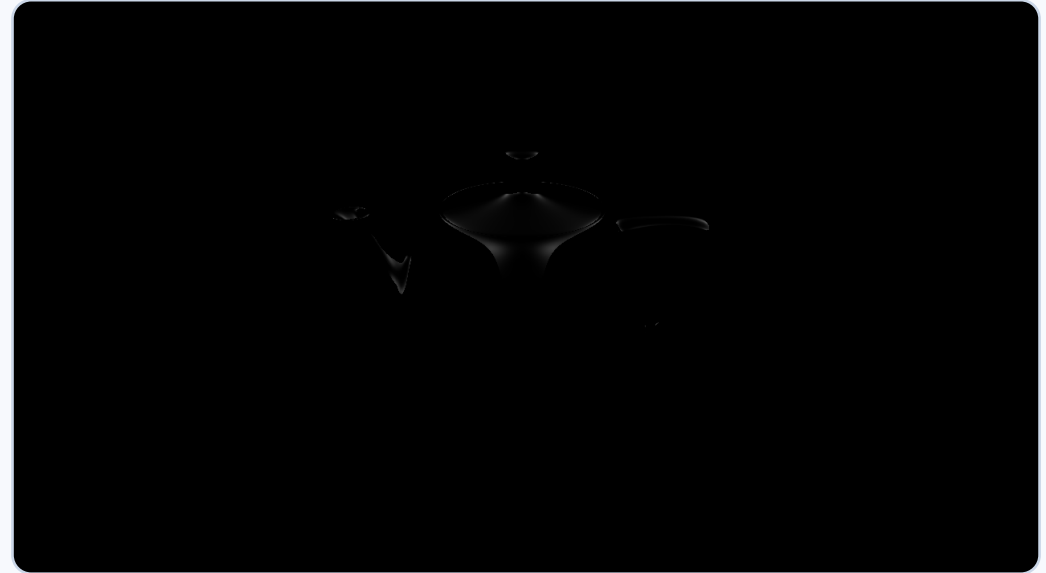
- Stable hit: sample the previous normal-color HDR frame at the refined UV.
- Off-screen or no hit: contribute no SSR; IBL still supplies environment reflection.
- Unstable crossing: reject rather than displaying a high-contrast silhouette leak.
- The history source avoids same-target feedback, but camera or object motion can expose one-frame lag. Debug-only frames are not promoted into that history.

Confidence, composition, and failure modes

The buffer is a partial specular source; IBL remains responsible for missing environment reflection



SSR debug: reflected color with confidence made visible.



Specular IBL debug: environment contribution that complements SSR.

CONFIDENCE PRODUCT

- Roughness fades toward SSRMaxRoughness.
- Edge fade hides rays near viewport boundaries.
- Distance fade suppresses very long marches.
- Local depth stability rejects discontinuity-prone hits.
- Lighting then applies Fresnel and SSRIntensity.

UNRECOVERABLE INFORMATION

- Off-screen objects cannot reflect.
- Occluded geometry behind the first depth layer is unavailable.
- Transparent objects neither appear as SSR hit geometry nor receive SSR in this pass.
- No Hi-Z acceleration, temporal accumulation, denoising, or multi-ray glossy integration is implemented.

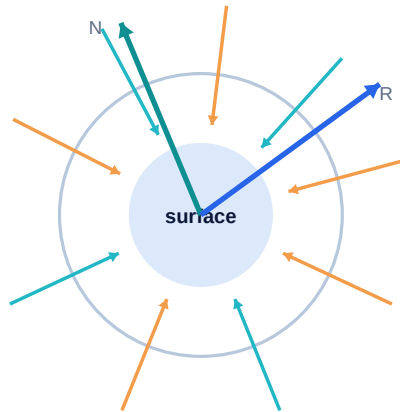
SECTION 04

Image Based Lighting

Treat an HDR environment as radiance arriving from every direction, then evaluate how diffuse and microfacet materials respond.

From an environment image to incident light

The environment replaces many explicit distant lights with a directional radiance function



DIFFUSE

Broadly averages incoming radiance over the normal-facing hemisphere. The result changes slowly with direction.

SPECULAR

Concentrates radiance around reflection direction R. Roughness widens the lobe; Fresnel strengthens grazing reflection.

THE LIGHTING INTEGRAL

$L_o(N, V) = \text{integral over hemisphere:}$

$$L_i(L) * \text{BRDF}(N, V, L) * \max(\text{dot}(N, L), 0) d\Omega$$

$L_i(L)$ comes from the HDR environment in direction L. The BRDF decides how strongly that direction contributes to the outgoing view direction V. Numerically integrating this expression for every pixel would require many environment samples.

DIFFUSE RESPONSE

An ideal Lambertian IBL path first convolves the environment with the cosine-weighted hemisphere. The result is an irradiance map: a slowly varying value indexed by surface normal N. It is then multiplied by diffuse albedo and divided by pi as part of the diffuse BRDF.

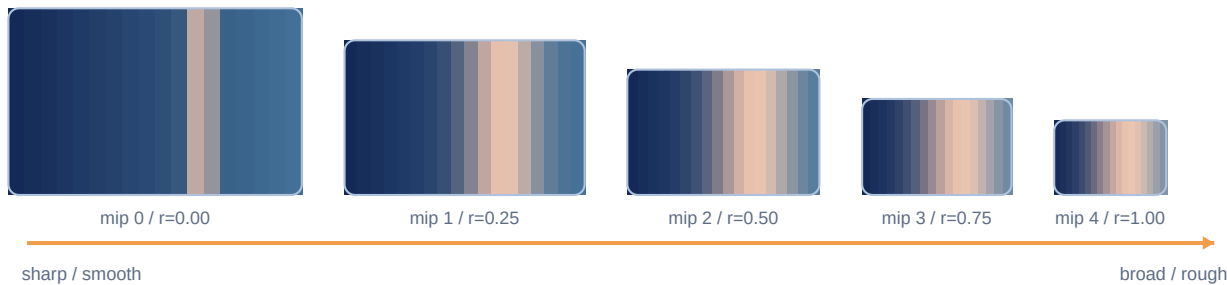
SPECULAR RESPONSE

A microfacet surface reflects a narrower or wider region around R. Roughness controls the lobe width, $N \cdot V$ controls the viewing geometry, and Fresnel controls how much energy reflects - especially near grazing angles. This is why specular IBL needs both filtered radiance and BRDF response data.

Roughness turns one direction into a distribution

Environment mips approximate the many radiance samples covered by a glossy reflection lobe

One reflection lookup selects a blur level from roughness



RUNTIME LOOKUP

```
R = reflect(-V, N)
lod = roughness * (mips - 1)
Li = envMap(R, lod)
```

- Low roughness selects a sharp mip and preserves small bright emitters.
- High roughness selects a blurrier mip that averages a wider angular neighborhood.
- The latitude-longitude seam is wrapped and clamped by mip-scaled texel margins.

IDEAL PREFILTER

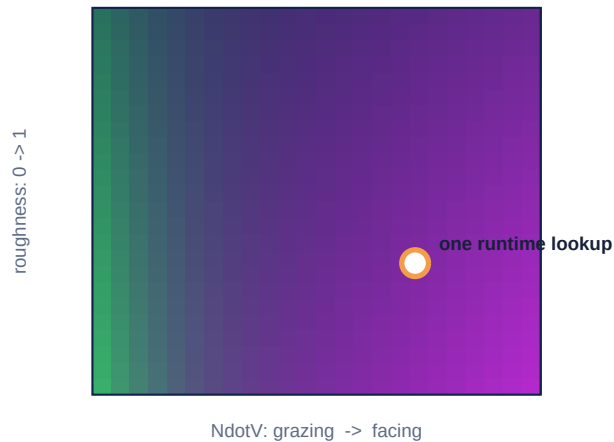
A production split-sum pipeline prefilters the environment with the same GGX normal-distribution model used by the BRDF. Each roughness level then represents the radiance expected under that microfacet lobe. This is more accurate than ordinary image reduction because bright features spread according to surface roughness, not only texel footprint.

WHAT THIS RENDERER DOES

`ReLoadEnvMap()` asks OpenGL to generate a standard mip chain and uses trilinear sampling. Those levels are box-filtered image mips, not GGX-prefiltered radiance. The shader still maps roughness to LOD, which gives useful blur, but highlights can lose energy or spread differently from the BRDF model. This distinction matters when interpreting the BRDF LUT: the LUT is physically motivated, while its radiance input is approximate.

The BRDF LUT: a response table, not a reflection image

It replaces hundreds of per-pixel microfacet samples with one 2D lookup returning two coefficients



LUT OUTPUT

RG = (A, B)

response = $F_0 * A + B$

WHAT IT DOES NOT STORE

No environment color, normal, reflection direction, or material albedo. It stores only the integrated BRDF response for view angle and roughness.

WHY TWO CHANNELS?

Schlick Fresnel is affine in the material's normal-incidence

$$\begin{aligned} F_c &= (1 - V_{dotH})^5 \\ F &= F_0 + (1 - F_0) * F_c \\ &= F_0 * (1 - F_c) + F_c \end{aligned}$$

after integration:
response = $F_0 * A + B$

A accumulates the part multiplied by F_0 ; B accumulates the grazing-angle Fresnel remainder. The LUT stores A in red and B in green for every (NdotV, roughness) pair.

OFFLINE / STARTUP

- Render a 256x256 fullscreen target.
- $X = NdotV$; $Y = \text{roughness}$.
- For each texel, importance-sample 1024 GGX half-vectors with a Hammersley sequence.
- Accumulate geometry visibility into A and B, then divide by sample count.

RUNTIME

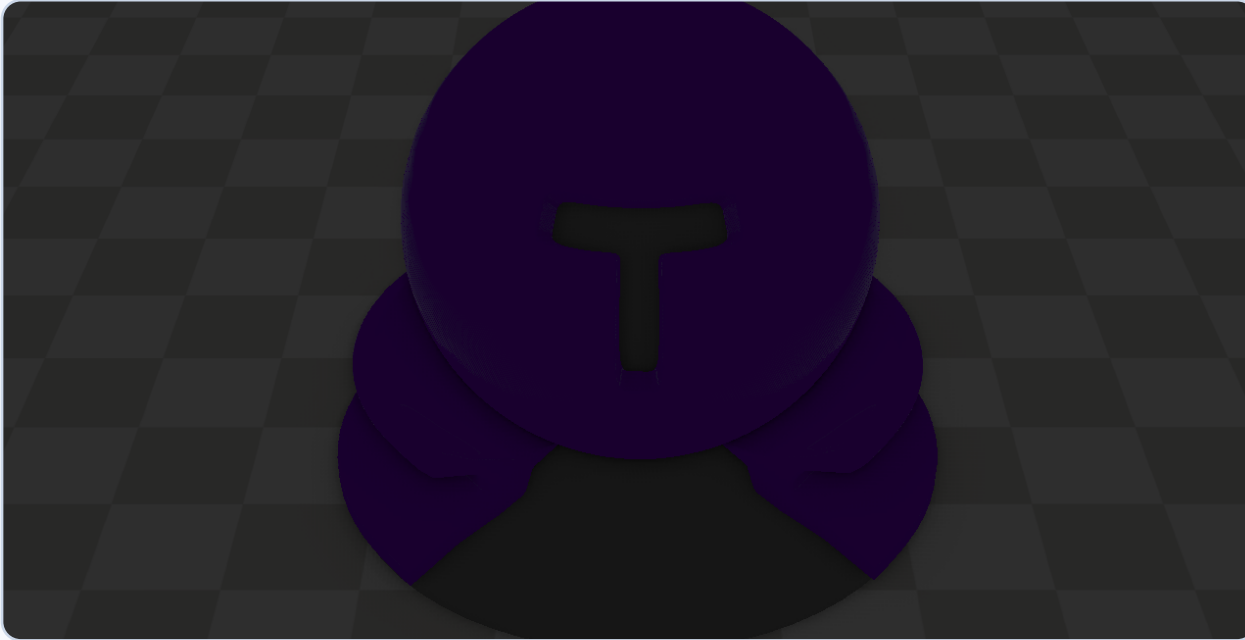
- Compute NdotV and roughness from the G-buffer.
- Fetch A, B = BRDFLUT(NdotV, roughness).
- Compute the material Fresnel color from F_0 , roughness, and view angle.
- Multiply filtered environment radiance by $F * A + B$.

FUNCTIONAL PURPOSE

The environment lookup answers *what radiance is present around R*? The BRDF LUT answers *what fraction of that filtered radiance reaches V after microfacet masking, geometry, and Fresnel response*? Neither lookup is complete alone.

How the current shader assembles IBL

A precise reading of fragment_DeferredLighting.glsl, including its deliberate approximations



Combined deferred IBL + SSR regression scene, tonemapped from the committed HDR baseline.

SPECULAR PATH

- Build F_0 from dielectric reflectance or metallic albedo.
- Reflect $-V$ around N and sample the environment at roughness-selected LOD.
- Sample the LUT with $(N \cdot V, \text{roughness})$.
- Use a roughness-aware Schlick Fresnel color, then evaluate $\text{prefiltered} * (F * A + B)$.
- Apply IBL intensity, max-roughness fade, and a light AO modulation.

DIFFUSE PATH

The shader samples the environment along N at $\min(4, \text{maxMip})$, then multiplies by albedo, non-metallic weight, AO, and 0.35. This is a blurred-environment approximation, not cosine-convolved irradiance.

MATERIAL CONTROLS

Roughness drives LOD and LUT Y . $N \cdot V$ drives LUT X . Metallic moves F_0 toward albedo and suppresses diffuse IBL. Reflectance sets dielectric F_0 as $0.16 * \text{reflectance}^2$. Environment rotation offsets longitude.

IBL + SSR

IBL is stable and sees all environment directions but no local objects. SSR contributes visible local scene detail. The current shader adds both; confident SSR does not replace IBL, so tuning their intensities is an artistic balance rather than an energy-conserving merge.

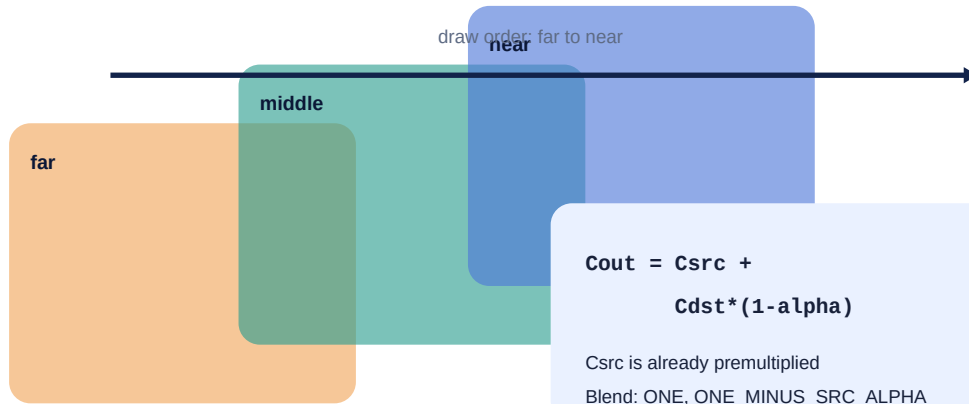
SECTION 05

Transparency

Keep multiple surface layers outside the single-hit G-buffer, sort them back to front, and composite premultiplied radiance.

Sorted forward shading over opaque color

A pragmatic quality-first approach, not order-independent transparency



CLASSIFICATION

- Alpha **MASK** stays opaque and discards below its cutoff.
- Alpha **BLEND** enters the transparent draw list.
- Any material with transmission above the threshold also enters the transparent list, even when its alpha mode says opaque.
- Transparent meshes are omitted from the G-buffer and shadow-map draw lists.

ORDERING

Instances are sorted every frame using the view-space depth of transformed mesh-bounds centers. Within each transparent mesh, triangle centers are also transformed, sorted back to front, and used to rewrite the EBO before drawing. This stabilizes shell-like glass meshes at a deliberate CPU and upload cost.

RENDER STATE

- Attach opaque depth; test with LEQUAL; disable depth writes.
- Cull back faces, shade with the forward PBR light path, and receive existing shadow maps and environment IBL.
- Blend with GL_ONE, GL_ONE_MINUS_SRC_ALPHA because shader RGB is premultiplied.

Coverage, premultiplication, and ordering

The blend equation is exact for an ordered layer stack; the hard part is finding that order

PREMULTIPLIED COMPOSITION

```
srcRGB = shadedRadiance * alpha
Cout = srcRGB + Cdst * (1 - alpha)
Aout = alpha + Adst * (1 - alpha)
```

Alpha represents the layer's coverage or opacity. Because RGB already includes alpha, the source blend factor is one. This avoids colored fringes around filtered texture edges and composes repeated layers associatively when their order is correct.

For transmissive materials, reflection and direct specular can remain visible even when body opacity is low: alpha is not a universal dimmer for every optical term. The shader packages the final radiance and coverage together before blending.

WHY DEPTH TEST, BUT NO DEPTH WRITE?

- Opaque depth rejects transparent fragments hidden behind the nearest opaque surface.
- A transparent layer must not replace that depth: farther transparent layers still need to pass and blend later in the back-to-front sequence.
- Back-face culling leaves one visible surface per closed shell direction. It reduces ordering ambiguity but also prevents true front/exit-interface transport through a glass volume.
- Each draw receives forward PBR lighting because its material data was never stored in the opaque G-buffer.

TWO SORTING LEVELS

The renderer first sorts mesh instances by transformed bounds-center depth. It then sorts triangles within a mesh by transformed triangle-center depth and rewrites the index order. Both are object-space approximations to a per-pixel visibility problem, but the second level greatly improves curved shells and overlapping faces within one asset.

WHERE SORTING FAILS

Intersecting polygons can form a cyclic order: A is behind B in one region and in front elsewhere. Large triangles can also span the camera so their centers give the wrong local order. Equal or rapidly changing depths may pop. No single global triangle order can solve these cases exactly.

HOW OIT WOULD DIFFER

Order-independent techniques retain or approximate multiple fragments per pixel. Depth peeling is accurate but uses repeated passes; per-pixel lists need variable storage; weighted blended OIT is fast but approximate. The current sorted path instead favors conventional blending, predictable memory, and good quality for non-intersecting glass.

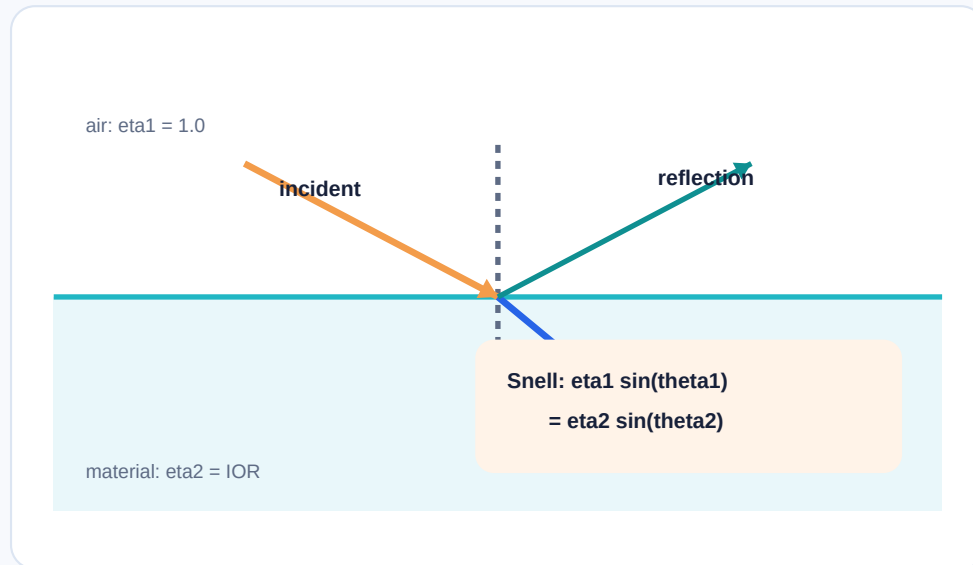
SECTION 06

Refraction

Bend a transmitted ray by index of refraction, trace it against the opaque screen-space scene, and fall back gracefully when information is missing.

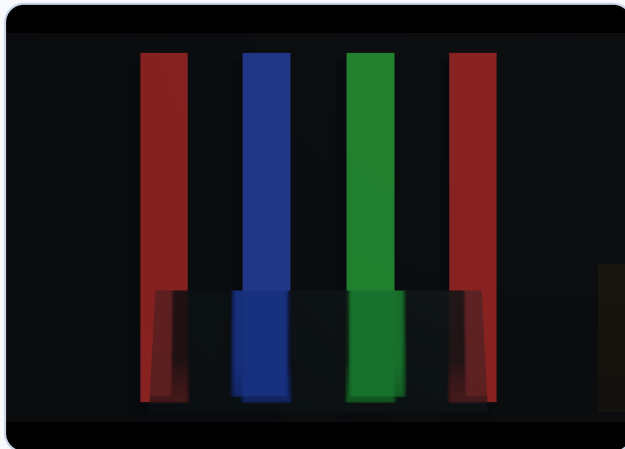
One interface, one opaque snapshot

Screen-space refraction reuses the SSR tracer but chooses different fallbacks and composition

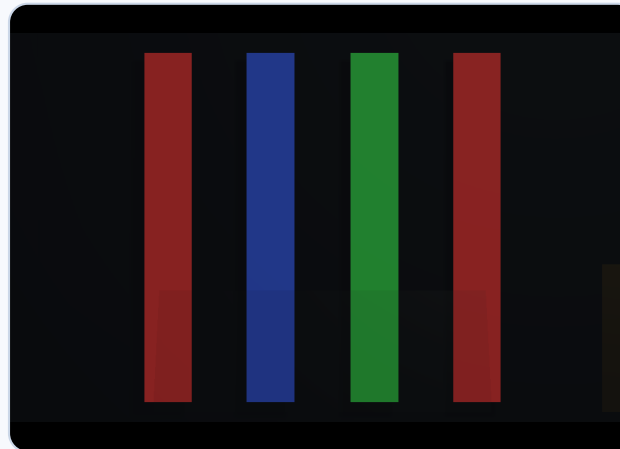


CURRENT ALGORITHM

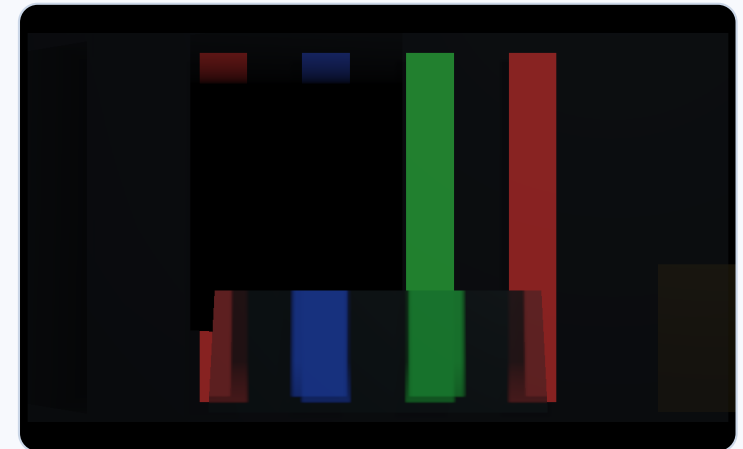
- Compute `refract(-V, N, 1/IOR)`; if it collapses, use reflection as the total-internal-reflection fallback.
- Trace against opaque position and depth using pixel stride, binary refinement, thickness, and depth stability.
- Sample a mipmapped copy of current opaque HDR color. LOD is $\text{roughness}^2 * (\text{mipCount} - 1)$.
- On genuine off-screen/no-hit status, sample the environment; on unstable hits, fade to the undisplaced opaque pixel.



Enabled



Refraction disabled: v1 transmission fallback



Environment fallback and IOR 1.0 coverage

Fresnel chooses reflection versus transmission

IOR bends the ray; Fresnel and tracer status decide which radiance source reaches the pixel

FROM IOR TO ENERGY SPLIT

```
eta = 1.0 / IOR
T = refract(-V, N, eta)
F0 = ((IOR - 1) / (IOR + 1))^2
F = F0 + (1 - F0) * (1 - NdotV)^5
```

Snell's law determines the transmitted direction T. The Schlick approximation estimates the reflected fraction F: most dielectric light transmits near normal incidence, while reflection grows toward grazing angles. The remaining fraction, 1-F, is available to transmission before material tint and transmission strength.

COMPOSITION IN THIS PASS

```
transmitted = sampledScene * albedo
* SpecTrans * (1 - F)
reflected = directSpecular + environmentSpecular
outRGB = premultiply(transmitted + reflected, coverage)
```

The opaque snapshot supplies background radiance without a read/write feedback loop. Roughness selects a blurrier snapshot mip, approximating a broader transmission lobe. Reflection remains a surface response; transmission samples what appears behind the surface in the available screen-space representation.

FALLBACK DECISION TREE

- **IOR approximately 1** -> Use the undisplaced opaque pixel.
- **Stable traced hit** -> Sample the refined refracted UV.
- **Off-screen / no hit** -> Use environment radiance when available.
- **Unstable depth** -> Fade toward the undisplaced opaque pixel.
- **Total internal reflection** -> Use the reflected direction instead.

WHAT A SINGLE INTERFACE OMITS

- A closed object should refract on entry and again on exit; this pass traces only the first air-to-material event.
- There is no geometric thickness, Beer-Lambert absorption, dispersion by wavelength, internal reflection path, or volume scattering.
- Only the opaque snapshot is visible through glass. Another transparent object cannot become a refracted hit layer.
- Consequently this is a convincing real-time image effect, not a physically complete simulation of transport through a solid dielectric.

Interactions, controls, and visibility limits

Choose parameters by the information each technique owns, not by treating every artifact as noise

Technique	Reads	Cannot see	Primary controls
Shadows	light depth + world point	transparent attenuation	resolution, bias, far, caster limit
SSAO	opaque normal/position/depth	hidden or transparent geometry	radius, bias, intensity, kernel, blur
SSR	opaque G-buffer + previous color	off-screen/occluded/transparent	steps, stride, bias, distance, thickness, fade
IBL	HDR environment + BRDF LUT	local scene objects	intensity, max roughness, environment rotation
Transparency	opaque depth + lights + env	order-independent intersections	enable; material opacity/transmission
Refraction	opaque depth/position/current color	farther transparent layers	steps, stride, bias, distance, thickness, edge fade

DEBUG VIEWS

Test5 exposes shadows, SSAO, specular IBL, material parameters, SSR, direct diffuse, and direct specular. Use one buffer at a time to isolate whether an artifact originates in geometry, visibility, shading, or composition.

COST SHAPE

Shadow cost scales with selected casters and shadow faces; SSAO with kernel size; SSR/refraction with traversed steps; transparency with visible transparent triangles plus per-frame sorting and EBO uploads; the BRDF LUT is generated once.

Debugging map, code entry points, and references

Start with the buffer that owns the missing information

SYMPTOM TO FIRST CHECK

- Detached or stippled shadows -> bias, far plane, map resolution, caster selection.
- Dark halos at silhouettes -> SSAO radius/range and blur depth weights.
- Reflection holes or bands -> SSR stride, thickness, depth discontinuities, viewport edges.
- Overly dull rough reflections -> ordinary env mip approximation and roughness fades.
- Glass triangle popping -> sorting ambiguity or intersecting transparent meshes.
- Refraction snapping to sky -> screen-space miss classification and edge confidence.

CODE ENTRY POINTS

```
Source/src/DeferredRenderer.cpp
Source/src/RenderSettings.h
Shaders/Shadows.glsl
Shaders/fragment_SSAO.glsl
Shaders/fragment_SSAOBlur.glsl
Shaders/fragment_SSR.glsl
Shaders/ScreenSpaceTrace.glsl
Shaders/fragment_DeferredLighting.glsl
Shaders/fragment_DeferredTransparent.glsl
Shaders/fragment_BRDFLUT.glsl
```

MINI GLOSSARY

- **G-buffer:** per-pixel geometric/material attributes.
- **PCF:** average of neighboring depth comparisons.
- **F0:** Fresnel reflectance at normal incidence.
- **IOR:** index of refraction.
- **Premultiplied:** RGB already contains alpha/coverage.
- **OIT:** order-independent transparency.

FOUNDATIONAL AND TECHNICAL REFERENCES

- [1] L. Williams, *Casting Curved Shadows on Curved Surfaces*, SIGGRAPH 1978. doi.org/10.1145/965139.807402
- [2] M. Mittring, *Finding Next Gen: CryEngine 2*, SIGGRAPH 2007 course. advances.realtimerendering.com/s2007
- [3] M. McGuire and M. Mara, *Efficient GPU Screen-Space Ray Tracing*, JCGT 3(4), 2014. jcgt.org/published/0003/04/04/
- [4] B. Karis, *Real Shading in Unreal Engine 4*, SIGGRAPH 2013 course notes.
- [5] T. Porter and T. Duff, *Compositing Digital Images*, SIGGRAPH 1984. doi.org/10.1145/800031.808606
- [6] Pharr, Jakob, and Humphreys, *Physically Based Rendering*, 4th ed., specular reflection and transmission. pbr-book.org
- [7] Khronos OpenGL documentation, blending factors and premultiplied-alpha setup. wikis.khronos.org/opengl/Draw_Buffer_Blend